

Extended Abstract

Motivation Pattern recognition is fundamental to reasoning and intelligence. Stronger pattern-recognition abilities result in improved performance (in both humans and LLMs) on important abstract-reasoning heavy domains like math. So, given the importance of pattern recognition, we seek to use RL to improve the pattern-recognition abilities of an LLM. Specifically, we use the game rock-paper-scissors (RPS) as the medium through which we can improve pattern recognition using RL. RPS allows for simple patterns encoded in the opponent’s moves and also gives immediate reward feedback.

Method We break our RPS task into 3 subtasks of increasing difficulty. Specifically we vary the pattern the agent plays RPS against; fixed-length-2, fixed-length-3, and random-length-2 opponent patterns. We train two agents and compare them. First, we train GPT-2 Small (124 M parameters) with a frozen backbone augmented by LoRA adapters and optimized via Proximal Policy Optimization (PPO), and second, a tiny, single-layer model trained from scratch only on the RPS task. Both share the same tokenizer, observation format, and reward schedule, and hyper-parameter sweeps confirm that their relative performance is robust to tuning.

Implementation Training employs PPO with a clip ratio of 0.2, generalized-advantage estimation ($\gamma = 0.99$, $\lambda = 0.95$), LoRA settings $r = 8$, $\alpha = 32$, dropout = 0.05, and gradient-norm clipping at 0.5. Thirty-two parallel environments collect 256-step rollouts, allowing the full run to complete in under two GPU-hours on a single A100.

Results The finetuned GPT-2 model wins 98 % of games on length-2 cycles but falls to 72 % on length-3 and 61 % when patterns are randomly permuted. The one-layer baseline secures 100 % on both deterministic tasks and retains 96 % against random-length-2 opponents despite using roughly 100× fewer parameters. Learning curves reveal GPT-2 plateauing early while the baseline continues to improve throughout training.

Discussion That the tiny, single hidden-layer model outperforms fine-tuned GPT-2 Small is certainly surprising. Still, our explanation is that the tasks are simple enough that they can be learned by a small model. GPT-2, however, is large enough that RL isn’t sufficient to shift its abilities so significantly. It’s language-heavy inductive biases can’t adapt easily to abstract pattern-recognition tasks. But it’s also too small to have baked in pattern-recognition abilities

Conclusion Small LLMs struggle with basic pattern-recognition. It’s not a capability they build strongly during pre-training but its also difficult for them to improve upon using RL. Going forward we would use SFT warmup to help the LLM perform better on the task.

Improving Pattern Recognition Using RL Through Rock-Paper-Scissors

James Poetzscher

Department of Computer Science
Stanford University
jamesp25@stanford.edu

Shubh Khanna

Department of Computer Science
Stanford University
shubh@stanford.edu

Siddharth Sharma

Department of Computer Science
Stanford University
sldrsh@stanford.edu

Abstract

Pattern recognition underlies many reasoning tasks that state-of-the-art large language models (LLMs) still find challenging. We investigate whether reinforcement learning (RL) can sharpen pattern-recognition skills in LLMs by fine-tuning GPT-2 on a family of rock-paper-scissors (RPS) games whose optimal policy requires inferring and countering deterministic action cycles. We introduce a lightweight text-based RPS environment that emits dense rewards at every move and supports three difficulty tiers: fixed-length-2, fixed-length-3, and random-length-2 opponent patterns. GPT-2 (124 M parameters) is adapted with LoRA and optimized with Proximal Policy Optimization, while a bespoke one-layer network trained from scratch serves as a size-matched baseline. Experiments over 256 K episodes reveal that GPT-2 attains a 98 % win rate on fixed-length-2 cycles but falls to 72 % on length-3 and 61 % once random shuffling is introduced. In contrast, the 1-layer model achieves perfect play on both deterministic settings (100 %) and maintains 96 % against random-length-2 patterns. Analysis suggests that (i) GPT-2’s frozen backbone and language-centric inductive bias hinder credit assignment for longer temporal dependencies, and (ii) uniformly trainable, low-capacity architectures adapt more coherently to dense RL signals. Our findings highlight a surprising trade-off: sheer parameter count and pre-training do not guarantee superior pattern-recognition once tasks depart from natural language, whereas small task-specific networks can learn robust strategies with minimal compute. The proposed RPS code is released to push further work on targeted RL fine-tuning for reasoning.

1 Introduction

Pattern recognition is core to abstract reasoning and intelligence, generally. Domains like math and programming rely heavily on pattern recognition as do virtually all other intellectually valuable and challenging tasks. Improving pattern recognition in LLMs is therefore an incredibly important aspect of furthering their capabilities. Much recent research has focused on improving general reasoning ability, especially using RL at post-training to allow for powerful inference time scaling Snell et al. (2024). Of course, improving overall reasoning, as these methods do, will necessarily improve pattern recognition. But, we were interested in whether we could *specifically* improve pattern recognition at post-training. That is, fine-tune an LLM using RL with the explicit goal of improving pattern recognition. We expected that doing so would, given the specific focus, improve pattern recognition

more than the standard RL for reasoning method. At the same time, we expected that successfully improving pattern recognition in this manner would lead to strong downstream improvements on important reasoning domains like math and coding.

Specifically, we chose to fine-tune a small open-weight LLM on a simple pattern recognition task. In choosing the task we wanted to ensure that abstract, general pattern recognition was the central focus of the task such that if a model were to perform well on the specific task, this would be a strong signal that its pattern recognition abilities had improved rather than some other ability associated with the task. Tasks that focus heavily on language—such as logic word puzzles or riddles—weren’t ideal; improvement on such a task could indicate improvement in pattern recognition and abstract reasoning but it also could simply indicate a better grasp of various puzzle-specific language related intricacies that don’t confer much of an improvement on general downstream reasoning tasks. We focused, then, on selecting tasks where patterns could be encoded very simply (e.g. using only a few integers) such that understanding and recognizing the patterns themselves, rather than the syntax they are wrapped in was the core challenge. We also wanted to choose a task that was realistic for a small open-weight LLM. The limited pre-train compute relative to larger models and smaller param count, means that for a model like GPT-2, the model’s baseline pattern recognition ability is almost non-existent (in contrast to a much larger model like GPT-4, where post-training is a means to improve its already quite impressive pattern recognition abilities) Wei et al. (2022); Mirchandani et al. (2023). We saw simple adversarial games, in which opponent play is deterministic and easily encoded as an integer-based pattern as the most effective way to satisfy the various goals we had for this task. And specifically, rock-paper-scissors (RPS) represented the perfect medium.

At a high-level, we took GPT-2 Small and fine-tuned it using RL on our RPS task. The task required our agent to play RPS against an opponent—a deterministic pattern of rock, paper and scissors, either fixed across all episodes of train and test, or randomly sampled for each episode. Each episode consisted, then, of the opponent making moves according to its pattern, and our agent, given access to the previous moves made by itself and the opponent that episode, attempts to win. The rules of the game match those of standard rock, paper, scissors, and so maximizing its win rate requires the agent to decipher the opponents pattern from the state history and then exploit the optimal counterplay.

We also trained a tiny single hidden layer model from scratch, using RL, on the objective as a gauge of the tasks difficulty and as a benchmark against which we could compare GPT-2 small’s performance.

2 Related Work

Much of the recent work on LLMs has focused on improving their reasoning capabilities through RL post-training. Specifically, models like OpenAI’s o3, Anthropic’s Claude 3.7 and 4.0, and DeepSeek’s R1 reflect significant improvements in reasoning by using RL at post-training DeepSeek-AI (2025); OpenAI (2025). With this increased focus on, and advances in, reasoning, a few recent papers have also attempted to improve the reasoning capabilities of LLMs by fine-tuning them on very specific tasks. This is in contrast to the much more robust, costly, general RL-for-reasoning methods employed by large labs at post-training. Many of these tasks have focused on improving domain specific abilities like math or competitive programming Ding et al. (2025); Wang et al. (2025).

While these papers focus on using RL to improve reasoning in a more compute-efficient way than the large-lab, inference time scaling methods, they focus on improving reasoning on a downstream task of interest by fine-tuning on that downstream task itself. We diverge from this approach, choosing to fine-tune on general pattern recognition with the expectation that true pattern-recognition improvements would generalize to improvements on an array of downstream tasks. Of course, it’s likely that by training on pattern-recognition itself, the improvements on any single downstream task wouldn’t be as strong as those attained by using RL directly on that task. We think this is a worthwhile tradeoff and makes sense as a sort of intermediary to boost general pattern recognition while more domain specific methods can later be implemented in addition.

However, recently there have been a few papers that align with our approach, fine-tuning LLMs using RL to improve pattern-recognition with the expectation that this should generalize downstream.

“Logic-RL: Unleashing LLM Reasoning with Rule-Based Reinforcement Learning” is a recent paper that shares much in common with our approach Xie et al. (2025). The authors use RL to improve the pattern-recognition and logical reasoning abilities of LLMs. Specifically, they opt to use synthetically

generated Knight and Knave puzzles as their medium; puzzles which encode a series of statements by the various characters and task the agent with deducing whether, given the statements and what would be required for all statements to be logically consistent, a character is telling the truth (and therefore a knight) or lying (and therefore a Knave). The authors focus on using RL to improve Qwen2.5-7B-Instruct-1M. They then evaluate how the fine-tuned model performs relative to the baseline on the AIME and AMC math exams, finding strong improvements after training on only 5,000 of these synthetic Knight and Knave puzzles.

While we think this approach is directionally correct, focusing on higher level causal variables associated with intelligence (logical reasoning and pattern recognition) rather than training directly on domain specific tasks, we think the specificity and heavy language focus of Knight and Knave puzzles likely hinders the generalized pattern recognition improvements. This, then, leads to less downstream improvements on reasoning tasks compared to training on similarly rigorous logic puzzles and pattern recognition tasks where the syntax and patterns themselves were general enough (e.g. basic integer patterns) that more of the compute dedicated towards learning the task is being put towards learning aspects of the game that generalize rather than non-generalizable aspects of the game like rules, syntax and

We also felt that the reward function in the Knight and Knave game was sparse and suboptimal, only assigning positive reward for correct formatting (to prevent reward hacking) and for getting the answer correct, or partially correct. On difficult, and often long puzzles, this results in positive reward being doled out less frequently than the model perhaps “deserves”; strong reasoning and making a minor mistake rendering the answer incorrect gets negative reward. With RPS, we have a dense reward assigned after every move (and each episode of training has 256 steps) based on whether the agent won that move.

Meanwhile, “Self-playing Adversarial Language Game Enhances LLM Reasoning,” uses an adversarial game, like we do, to improve reasoning Cheng et al. (2025). The authors take perhaps a slightly less direct approach to improving reasoning and pattern recognition; they train an attacker and a defender, where the attacker and defender have a conversation in which the attacker attempts to get the defender to say the target word, and the defender aims to guess the target word without saying it. While this approach does improve reasoning capabilities, as shown by performance improvements on various benchmarks, we feel that the approach is, again, overly language heavy, distracting the focus of training away from pure reasoning. Also, unlike our approach with RPS, where the opponent is not a model but a pattern generated from the environment, they train both the opponent and agent. While clever for their use case, we felt that training the opponent in addition to the agent for our RPS task would only complicate training without conferring any significant benefit; generating short (length between 2 and 6 moves) patterns is simple enough that training an opponent to generate “hard” patterns of such a format would be pointless—it wouldn’t improve the model’s reasoning ability.

3 Method and Experimental Setup

We train two different models as our agent. First, we used GPT-2 Small, a 124M parameter model. We froze the base weights, used LoRA, and attached a policy head mapping logits to moves and a value head mapping logits to state values. Next, to gauge the difficulty of the RPS task, and to evaluate the performance of GPT-2, we used RL to train from scratch a small model. Here, our network architecture consists entirely of a policy head and value head, both with two hidden layers.

Our goal is to use RPS as the medium through which our agent improves its pattern recognition abilities. Crucially, though, the specific task must be feasible for the agent to learn on—a significant challenge we faced as most tasks proved too difficult for GPT-2 to succeed at. We subdivide our RPS task into three subtasks of increasing difficulty. In task 1, we train our agent on a fixed, length-2 pattern, meaning that our agent is faced with an opponent who plays the same 2 moves across all episodes (both training and test-time episodes). In task 2, we do the same but with a fixed, length 3-pattern. Finally, in task 3, we generalize to random patterns of length 2, meaning that, each episode the environment samples a random length-2 pattern for the opponent to play against.

Each episode consists of the opponent sampling a pattern (or using the fixed pattern chosen ahead of the training run) and then, for each step in the episode, the opponent plays a move according to the pattern. So, if the pattern is [rock, paper], on odd number steps (assuming the first step is step 1), the opponent plays rock and on even number moves the opponent plays paper. At the same time, the

agent is fed the history of the past X moves—both the opponents’ moves and its own moves—as well as an indicator on the outcome for that move (win, draw, or loss). X , is a hyper-parameter that we chose to be 10 for our final training runs—this means the agent, on every step except for the first 10, can see multiple full sequences of the pattern played out in its history. Conditional on the state, then, the agent makes a move and we assign reward according to the standard rules of RPS: rock beats paper, paper beats scissors, scissors beats rock and if both players play the same move this is a draw.

Our reward function rewards wins with +1, losses with -1, and draws with -0.5. We found that this helped avoid a local minima trap where when we gave draws 0 reward, rather than -0.5, the model was more likely to stick to playing a single move every step; that is, if the fixed, length-2 pattern was [paper, scissors], if you play a single move every step you’re best off playing scissors (draw against scissors, and beat paper) so the agent would resort most often to this strategy. By penalizing draws we were better able to avoid this issue. We also experimented with assigning rewards only for getting sequences of moves right: $2(n - 1)$ points on a move where n represents the number of moves the agent has won in a row. But, this didn’t improve results, we suspect because credit assignment with this reward function was suboptimal.

3.1 Rock-Paper-Scissors Environment

We can more rigorously define our RPS environment. Assuming our task 2 definition of a length 3 fixed pattern, let $\mathcal{A} = \{0, 1, 2\}$ represent the action space where 0 = Rock, 1 = Paper, 2 = Scissors. This environment is defined by:

- **State space:** $S = \{s_t\}$ where $s_t = (h_t, t)$ contains history h_t and timestep t
- **Opponent strategy:** Fixed pattern $\mathbf{p} = [0, 1, 2]$ (Rock, Paper, Scissors cycle)
- **Reward function:** $R(a_t, o_t) = W_{a_t, o_t}$ where W is the modified win matrix:

$$W = \begin{pmatrix} -0.5 & -1 & 1 \\ 1 & -0.5 & -1 \\ -1 & 1 & -0.5 \end{pmatrix} \quad (1)$$

Note that ties in a penalty of -0.5 rather than 0, encouraging more aggressive play. The opponent’s action at timestep t is deterministic: $o_t = p_{t \bmod 3}$.

The state s_t is encoded as text and processed by a GPT-2 network:

$$s_t^{\text{ext}} = \text{“Step } t. \text{ H: } h_{t-k:t-1}\text{”} \quad (2)$$

where $h_{t-k:t-1}$ represents the last k history tuples (a_i, o_i, r_i) encoded as strings in the format $a_i o_i \text{sign}(r_i)$.

3.2 PPO Algorithm

For the policy and value networks, we use a shared GPT-2 backbone f_θ with LoRA adaptation and separate heads:

$$\pi_\theta(a|s) = \text{Categorical}(W_\pi f_\theta(s) + b_\pi) \quad (3)$$

$$V_\theta(s) = W_v f_\theta(s) + b_v \quad (4)$$

where θ includes the GPT-2 parameters (with LoRA), and head parameters (W_π, b_π, W_v, b_v) . The LoRA configuration uses: rank $r = 8$, alpha $\alpha = 32$, dropout = 0.05, target modules: attention and projection layers

We also employ Generalized Advantage Estimation (GAE). For trajectory $\tau = (s_0, a_0, r_0, s_1, \dots, s_T)$, we compute:

$$\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \quad (5)$$

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{T-t-1} (\gamma \lambda)^l \delta_{t+l} \quad (6)$$

where $\gamma = 0.99$ is the discount factor and $\lambda = 0.95$ is the GAE parameter.

The returns are computed as:

$$R_t = A_t^{\text{GAE}} + V_\theta(s_t) \quad (7)$$

Bootstrap values for terminal states are set to zero: $V_\theta(s_T) = 0$ when episode ends.

The core PPO update uses the clipped surrogate objective. For each minibatch:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (8)$$

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (9)$$

where $\epsilon = 0.2$ is the clipping parameter.

Note that our PPO also uses a multi-objective loss function. The total loss combines three objectives:

$$L(\theta) = L^{\text{CLIP}}(\theta) + c_1 L^{VF}(\theta) - c_2 S[\pi_\theta](s_t) \quad (10)$$

where:

$$L^{VF}(\theta) = \mathbb{E}_t [(V_\theta(s_t) - R_t)^2] \quad (11)$$

$$S[\pi_\theta](s_t) = \mathbb{E}_t \left[\sum_{a \in \mathcal{A}} \pi_\theta(a|s_t) \log \pi_\theta(a|s_t) \right] \quad (12)$$

with coefficients $c_1 = 0.5$ (value function weight) and $c_2 = 0.01$ (entropy weight).

3.3 Training Procedure

Our full PPO training procedure is featured below. Our full training runs were conducted on A100 NVIDIA GPUs and each run took under 2 hours for 256,000 steps (1,000 episodes of step size 256).

Algorithm 1 PPO for RPS Pattern Recognition with GPT-2

```

1: Initialize GPT-2 policy  $\pi_\theta$  and value function  $V_\theta$  with LoRA
2: for episode = 1, ...,  $N_{\text{episodes}}$  do
3:   Reset environment with pattern [0, 1, 2]
4:   for step = 1, ...,  $T_{\text{episode}}$  do
5:     Collect transition  $(s_t, a_t, r_t, s_{t+1})$ 
6:     if step mod  $T_{\text{rollout}} = 0$  then
7:       Compute advantages  $\{A_t\}$  using GAE
8:       Normalize advantages:  $A_t \leftarrow \frac{A_t - \mu_A}{\sigma_A}$ 
9:       for ppo_epoch = 1, ...,  $K$  do
10:        for each minibatch  $\mathcal{B} \subset \mathcal{D}$  do
11:          Compute ratio  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 
12:          Compute clipped loss  $L^{\text{CLIP}}(\theta)$ 
13:          Compute value loss  $L^{VF}(\theta)$ 
14:          Compute entropy loss  $S[\pi_\theta]$ 
15:          Update  $\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$ 
16:          Apply gradient clipping with max norm 0.5
17:        end for
18:      end for
19:    end if
20:  end for
21: end for

```

Table 1: GPT-2 Finetune performance against patterned opponents

Opponent pattern	Win rate	Avg. reward
Fixed length 2 (e.g. [R,P])	98%	251.5
Fixed length 3 (e.g. [R,P,S])	72%	128.5
Random length 2	61%	80.6

4 Results

We benchmarked two architectures on a family of rock–paper–scissors (RPS) pattern-exploitation tasks.

(i) Finetuned GPT-2: a 124 M-parameter GPT-2 checkpoint updated with Proximal Policy Optimization (PPO) for 256 K self-play episodes.

(ii) Naïve single-layer model (trained from scratch): a compact, one-layer model trained from scratch on the identical objective.

In every experiment an adversary produces a length- n action cycle and repeats it forever. A *fixed-length* n task is fully deterministic (the exact same sequence recurs), whereas a *random-length* n task permutes the n actions between games while keeping the period length unchanged.

GPT-2 (finetuned). The finetuned GPT-2 achieves strong but imperfect play on short, regular patterns. Its win rate is 98 % on fixed-length-2 cycles, falls to 72 % when the cycle extends to length 3 (a drop of 26 percentage points), and declines further to 61 % once stochastic shuffling is introduced at length 2. These results indicate a heavy reliance on memorizing short-horizon correlations; performance erodes when either temporal depth or randomness undermines that memorization.

Naïve single-layer model. Despite having roughly $\times 100$ fewer parameters, the naïve model attains perfect exploitation on both deterministic settings, winning 100 % of games at fixed lengths 2 and 3. When faced with random-length-2 patterns it still secures a 96 % win rate—only a modest four-percentage-point decline. The model’s inductive bias toward short, repetitive structures, coupled with end-to-end training, appears to confer robustness that the partially frozen GPT-2 does not share.

- **Difficulty scaling.** Both models excel on the simplest task, but GPT-2 degrades dramatically as the temporal structure lengthens or becomes less regular.
- **Parameter efficiency.** The one-layer baseline outperforms the 124 M-parameter GPT-2 across all conditions, demonstrating that sheer capacity is insufficient for low-entropy adversarial games.
- **Practical takeaway.** Large language models can be competitive after finetuning, yet lightweight, task-specialised networks may deliver higher win rates and degrade more gracefully under stochastic perturbations.

In summary, our finetuned GPT-2 offers high—but not perfect—accuracy on very short, deterministic sequences, whereas the simpler architecture sustains perfect or near-perfect performance and exhibits superior generalization as the opponent’s behavior grows longer or more variable.

Table 2: RL-only model performance against patterned opponents

Opponent pattern	Win rate	Avg. reward
Fixed length 2 (e.g. [R,P])	100%	256
Fixed length 3 (e.g. [R,P,S])	100%	256
Random length 2	96%	241

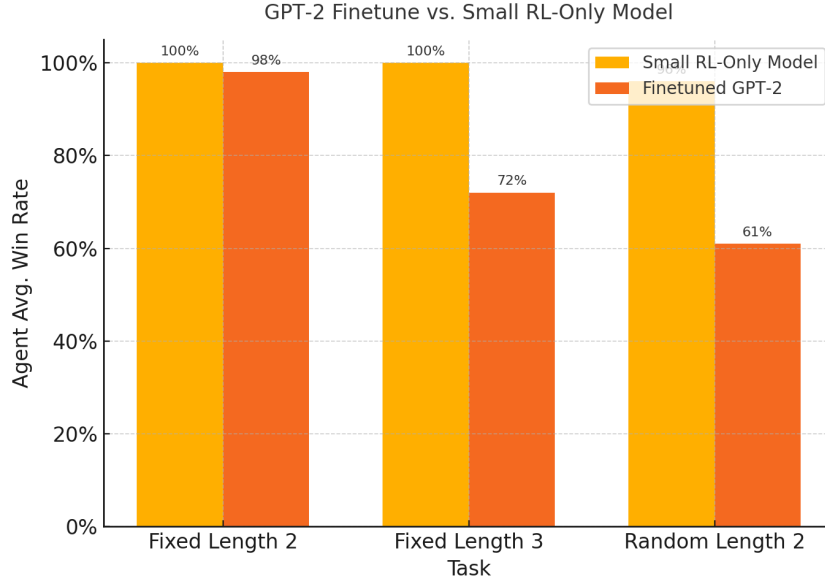


Figure 1: Average win rate of the finetuned GPT-2 versus a small RL-only model on three opponent patterns.

5 Discussion and Conclusion

We observe pronounced differences between the two learning paradigms we explored—*continual refinement* of a large, pre-trained model versus *bespoke training* of a compact architecture—and these disparities manifest not only in raw performance but also in sample efficiency, stability during optimization, and emergent generalization.

Although our PPO-based finetuning succeeds on trivial length-2 cycles, the model plateaus rapidly as temporal structure becomes richer. Precisely *why* GPT-2’s inductive biases hurt here is multifaceted:

- **GPT-2 is a naive model without broader skills:** Imbuing reasoning and pattern recognition capabilities into a simple model like GPT-2, which struggles at even basic language tasks, is difficult. GPT-2, unlike larger LLMs, hasn’t been able to generalize from basic language skills to broader pattern recognition and out of domain tasks. It’s thus difficult to finetune weights to add this additional capability while maintaining it’s current ability as a language model.
- **Sparse Signal:** A 1-layer model, while much tinier, has each update make a much larger difference across all parameters of the network. This could help align parameters with the objective, versus GPT-2’s parameters having too much inertia.

Conversely, the minimal 1-layer model, though many orders of magnitude smaller, learns near-optimal play across all regimes. A few factors appear decisive:

- **Dense, global updates.** Because every trainable tensor is directly exposed to the loss, even short training runs meaningfully reshape the entire hypothesis class. With fewer filters, gradients propagate without vanishing, and the model rapidly aligns all weights to the game-theoretic objective.
- **Lower variance during exploration.** The lightweight model’s action logits update cohesively, so the policy stabilises early and exploration concentrates around promising phases. GPT-2, by contrast, makes many near-zero shifts that keep its policy in a high-entropy limbo for far longer.

In this work, we strove to be parameter and compute efficient and hence limited our final experiments to GPT-2 (although we did experiment with larger models such as Qwen2.5 1.5B which performed similarly poorly). Our work shows that adapting small LLMs to pattern-recognition tasks is difficult using RL only methods. Toy models, solely trained for the task, perform well, confirming that our

RPS task isn't overly difficult. The large LLM backbones, meanwhile, require significant inertia to shift their capabilities, which our RL training wasn't able to provide. At the same time, they lack the base-line capabilities of their even larger, more powerful counterparts that could solve such problems trivially with in-context examples.

Going forward, we will focus on new techniques, beyond pure RL with PPO, in an effort to improve the performance of fine-tuned LLMs on the RPS task. Notably, we think using a supervised finetuning (SFT) warm up is a promising path for such improvement.

6 Team Contributions

- **James Poetzsch** Worked on creating environment and training code with focus on the more compact RL-only model. Helped make poster and write report.
- **Shubh Khanna** Worked on creating environment and training code with focus on evaluations and finetuning existing checkpoints. Helped make poster and write report.
- **Siddharth Sharma** Worked on creating environment and training code with a focus on implementing PPO from scratch. Helped make poster and write report.

Changes from Proposal We reduced the scope of the project, after realizing that the base task (using RL to learn RPS) was more challenging than we expected. We decided not to train the opponent and rather to simply generate opponent patterns randomly. We also chose not to evaluate on the downstream benchmarks we wanted to use initially, realizing that the model's struggles to learn the basic RPS task meant there would be no downstream improvement. Instead, we chose to train a small, RL only model to gauge how difficult the task was and how well our fine-tuned GPT-2 performed in comparison.

References

- Pengyu Cheng, Tianhao Hu, Han Xu, Zhisong Zhang, Zheng Yuan, Yong Dai, Lei Han, Nan Du, and Xiaolong Li. 2025. Self-playing Adversarial Language Game Enhances LLM Reasoning.
- DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.
- Fei Ding, Baiqiao Wang, Zijian Zeng, and Youwei Wang. 2025. Multi-Layer GRPO: Enhancing Reasoning and Self-Correction in Large Language Models.
- Suvir Mirchandani, Fei Xia, Pete Florence, Brian Ichter, Danny Driess, Montserrat Gonzalez Arenas, Kanishka Rao, Dorsa Sadigh, and Andy Zeng. 2023. Large Language Models as General Pattern Machines.
- OpenAI. 2025. Competitive Programming with Large Reasoning Models.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters.
- Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. 2025. Reinforcement Learning for Reasoning in Large Language Models with One Training Example.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models.
- Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. 2025. Logic-RL: Unleashing LLM Reasoning with Rule-Based Reinforcement Learning.